



# Analysis and Design of Algorithms



CP  
CS  
22  
3

رَبِّ اشْرَحْ لِي صَدْرِي وَيَسِّرْ لِي أَمْرِي وَاحْلُلْ عُقْدَةً مِنْ لِسَانِي يَفْقَهُوا قَوْلِي



O my Lord! Expand for me my chest [with assurance] and ease for me my task and untie the knot from my tongue that they may understand my speech. (Quran 20 : 25-28)



# *Analysis and Design of Algorithms*



# Algorithm Analysis

Recurrence  
(Formal Analysis)



## General Recurrence

- “It is an equation or inequality that describe a function in terms of its value on smaller input”
- $T(n) = a T(n/b) + f(n)$ 
  - $n$  = size of input
  - $a$  = no. of sub-problems
  - $b$  = value to yield  $n / b$  problem size
  - $a \geq 1$  and  $b > 1$



# Analysis and Design of Algorithms



## Introduction

- Informal Analysis
- Formal Analysis
  - Iterative Algorithms
    - RAM Model Analysis
  - **Recursive Algorithm**
    - Informal Analysis
    - Backwards Substitution Method
    - Substitution Method
    - Recursion Tree Method
    - Master Theorem Method

CP  
CS  
22  
3



# Analysis and Design of Algorithms



CP  
CS  
22  
3

## Merge Sort

MERGE-SORT( $A, p, r$ )

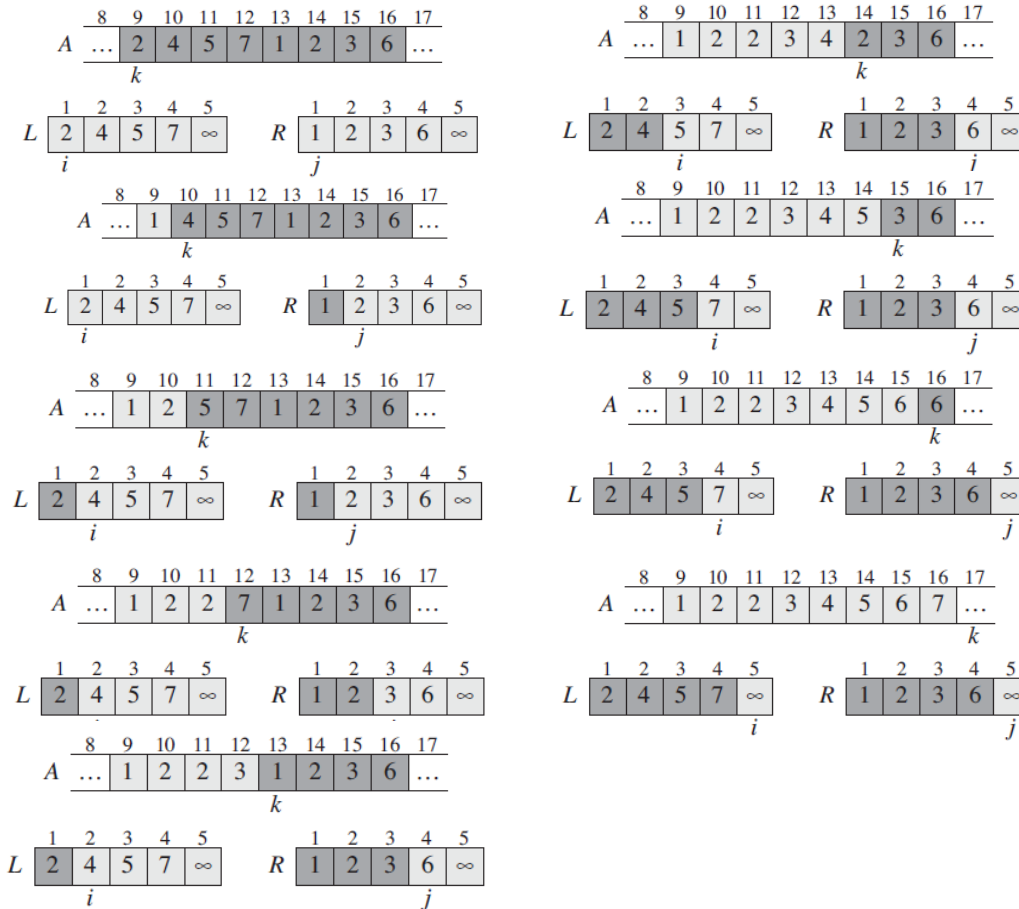
```
1  if  $p < r$ 
2       $q = \lfloor (p + r) / 2 \rfloor$ 
3      MERGE-SORT( $A, p, q$ )
4      MERGE-SORT( $A, q + 1, r$ )
5      MERGE( $A, p, q, r$ )
```

MERGE( $A, p, q, r$ )

```
1   $n_1 = q - p + 1$ 
2   $n_2 = r - q$ 
3  let  $L[1 \dots n_1 + 1]$  and  $R[1 \dots n_2 + 1]$  be new arrays
4  for  $i = 1$  to  $n_1$ 
5       $L[i] = A[p + i - 1]$ 
6  for  $j = 1$  to  $n_2$ 
7       $R[j] = A[q + j]$ 
8   $L[n_1 + 1] = \infty$ 
9   $R[n_2 + 1] = \infty$ 
10  $i = 1$ 
11  $j = 1$ 
12 for  $k = p$  to  $r$ 
13     if  $L[i] \leq R[j]$ 
14          $A[k] = L[i]$ 
15          $i = i + 1$ 
16     else  $A[k] = R[j]$ 
17          $j = j + 1$ 
```

$$\begin{aligned} T(n) &= \Theta(1) && \text{if } n = 1 \\ &= 2 T(n/2) + \Theta(n) && \text{if } n > 1 \end{aligned}$$

## Merge Sort



MERGE( $A, p, q, r$ )

```

1   $n_1 = q - p + 1$ 
2   $n_2 = r - q$ 
3  let  $L[1..n_1 + 1]$  and  $R[1..n_2 + 1]$  be new arrays
4  for  $i = 1$  to  $n_1$ 
5       $L[i] = A[p + i - 1]$ 
6  for  $j = 1$  to  $n_2$ 
7       $R[j] = A[q + j]$ 
8   $L[n_1 + 1] = \infty$ 
9   $R[n_2 + 1] = \infty$ 
10  $i = 1$ 
11  $j = 1$ 
12 for  $k = p$  to  $r$ 
13     if  $L[i] \leq R[j]$ 
14          $A[k] = L[i]$ 
15          $i = i + 1$ 
16     else  $A[k] = R[j]$ 
17          $j = j + 1$ 

```



# Analysis and Design of Algorithms



## Merge Sort - Recurrence

MERGE-SORT( $A, p, r$ )

1 if  $p < r$

2      $q = \lfloor (p + r)/2 \rfloor$

3     MERGE-SORT( $A, p, q$ )

4     MERGE-SORT( $A, q + 1, r$ )

5     MERGE( $A, p, q, r$ )

$$T(n) = \Theta(1) \quad \text{if } n = 1$$

$$= T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + \Theta(n) \quad \text{if } n > 1$$

$$T(n) = 2T(n/2) + \Theta(n)$$

CP  
CS  
22  
3



# Analysis and Design of Algorithms



## Substitution Method (Using Mathematical Induction)

- It has TWO steps
  - Guess the solution
  - Use **Mathematical Induction** to find the constant and show that solution works
- Powerful method to solve
- But demands practice
- Can solve either upper or lower bounds

CP  
CS  
22  
3





# Analysis and Design of Algorithms



## Merge Sort Recurrence

$$T(n) = \Theta(1) \quad \text{if } n = 1$$

$$= T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + \Theta(n) \quad \text{if } n > 1$$

$$T(n) = 2T(n/2) + \Theta(n)$$

- Upper Bound

$$T(n) = 2T(\lfloor n/2 \rfloor) + n$$

$$T(n) = O(n \lg n).$$

- Lower Bound

$$T(n) = 2T(\lceil n/2 \rceil) + n$$

$$T(n) = \Omega(n \lg n)$$

- Guess The Solution

$$T(n) = \Theta(n \lg n)$$

- Prove by Mathematical Induction

- Assumption:

- Any specific value of  $n$

- Prove for all values "n"

- $T(n)$

- Boundary Conditions

- $n = 1, 2, 3, 4, \dots$  etc

CP

CS

22

3



# Analysis and Design of Algorithms



## Merge Sort – Upper Bound

$$T(n) = 2T(\lfloor n/2 \rfloor) + n$$

**Guess Solution:**

$$T(n) = O(n \lg n).$$

$$T(n) \leq cn \lg n$$

**Prove using M. I. :**

**Step 01 : Assumption**

$$n = \lfloor n/2 \rfloor$$

$$T(\lfloor n/2 \rfloor) \leq c \lfloor n/2 \rfloor \lg(\lfloor n/2 \rfloor)$$

$$T(n) = \Theta(1) \quad \text{if } n = 1$$

$$= T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + \Theta(n)$$

$$\text{if } n > 1$$

**Step 02 : Prove for all “n”**

$$T(n) \leq 2(c \lfloor n/2 \rfloor \lg(\lfloor n/2 \rfloor)) + n$$

$$\leq 2(c \lfloor n/2 \rfloor \lg(n/2)) + n$$

$$= cn \lg(n/2) + n$$

$$= cn \lg n - cn \lg 2 + n$$

$$= cn \lg n - cn + n$$

$$\leq cn \lg n \quad c \geq 1$$

$$n_0 = 1$$



# Analysis and Design of Algorithms



## Merge Sort – Upper Bound

Recurrence:

$$T(n) = 2T(\lfloor n/2 \rfloor) + n$$

$$T(n) = \Theta(1) \quad n = 1$$

Solution:

$$T(n) = O(n \lg n).$$

$$T(n) \leq cn \lg n$$

Step 03 : Boundary Conditions

|          |                                    |               |
|----------|------------------------------------|---------------|
|          | $T(n)$                             | $O(n \lg n).$ |
|          | $2T(\lfloor n/2 \rfloor) + n \leq$ | $cn \lg n$    |
| $n = 1,$ | $2T(\lfloor 1/2 \rfloor) + 1 \leq$ | $c1 \lg 1$    |
|          | $2T(\lfloor 0 \rfloor) + 1 \leq$   | $c1 \cdot 0$  |
|          | $0 + 1 \leq$                       | $0$           |
|          | $1 \leq$                           | $0$           |
|          | $1 \leq 0$                         |               |

False

CP  
CS  
22  
3



# Analysis and Design of Algorithms



## Merge Sort – Upper Bound

Recurrence:

$$T(n) = 2T(\lfloor n/2 \rfloor) + n$$

$$T(n) = \Theta(1) \quad n = 1$$

Solution:

$$T(n) = O(n \lg n).$$

$$T(n) \leq cn \lg n$$

Step 03 : Boundary Conditions

$$\begin{array}{rcl} T(n) & & \\ 2T(\lfloor n/2 \rfloor) + n & \leq & cn \lg n \\ n = 2 & 2T(\lfloor 2/2 \rfloor) + 2 & \leq c2 \cdot \lg 2 \\ & 2T(1) + 2 & \leq c2 \cdot 1 \\ & 2 \cdot 1 + 2 & \leq 2c \\ & 4 & \leq 2c \\ & 4 & \leq 2c \end{array}$$

True for  $c \geq 2$

CP  
CS  
22  
3



# Analysis and Design of Algorithms



## Merge Sort – Upper Bound

Recurrence:

$$T(n) = 2T(\lfloor n/2 \rfloor) + n$$

$$T(n) = \Theta(1) \quad n = 1$$

Step 03 : Boundary Conditions

Solution:

$$T(n) = O(n \lg n).$$

$$T(n) \leq cn \lg n$$

$$T(n)$$

$$2T(\lfloor n/2 \rfloor) + n \leq cn \lg n$$

$$n = 3, \quad 2T(\lfloor 3/2 \rfloor) + 3 \leq c3 \cdot \lg 3$$

$$2T(\lfloor 1 \rfloor) + 3 \leq c3 \cdot (1.58)$$

$$2 \cdot 1 + 3 \leq (4.8) \cdot c$$

$$5 \leq (4.8) \cdot c$$

$$5 \leq (4.8) \cdot c$$

True for  $c \geq 2$

Hence proved

$$T(n) \leq cn \lg n$$



# Analysis and Design of Algorithms



## Merge Sort – Lower Bound

$$T(n) = 2T(\lceil n/2 \rceil) + n$$

**Guess Solution:**

$$T(n) = \Omega(n \lg n)$$

$$T(n) \geq cn \lg n$$

**Prove using M. I. :**

**Step 01 : Assumption**

$$n = \lceil n/2 \rceil$$

$$T(\lceil n/2 \rceil) \geq c \lceil n/2 \rceil \lg(\lceil n/2 \rceil)$$

$$T(n) = \Theta(1) \quad \text{if } n = 1$$

$$= T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + \Theta(n)$$

if  $n > 1$

**Step 02 : Prove for all “n”**

$$T(n) \geq 2(c \lceil n/2 \rceil \lg(\lceil n/2 \rceil)) + n$$

$$\geq 2(c \lceil n/2 \rceil \lg(n/2)) + n$$

$$= cn \lg(n/2) + n$$

$$= cn \lg n - cn \lg 2 + n$$

$$= cn \lg n - cn + n$$

$$\geq cn \lg n \quad \begin{matrix} c \leq 1 \\ n_0 = 1 \end{matrix}$$

CP  
CS  
22  
3



# Analysis and Design of Algorithms



## Merge Sort – Lower Bound

Recurrence:

$$T(n) = 2T(\lceil n/2 \rceil) + n$$

$$T(n) = \Theta(1) \quad n = 1$$

Step 03 : Boundary Conditions

Solution:

$$T(n) = \Omega(n \lg n)$$

$$T(n) \geq cn \lg n$$

$$\begin{aligned} T(n) &= 2T(\lceil n/2 \rceil) + n \geq cn \lg n \\ n = 1, \quad 2T(\lceil 1/2 \rceil) + 1 &\geq c1 \lg 1 \\ 2T(1) + 1 &\geq c1 \cdot 0 \\ 2 + 1 &\geq 0 \\ 3 &\geq 0 \\ 3 &\geq 0 \end{aligned}$$

True

CP  
CS  
22  
3



# Analysis and Design of Algorithms



## Merge Sort – Lower Bound

Recurrence:

$$T(n) = 2T(\lceil n/2 \rceil) + n$$

$$T(n) = \Theta(1) \quad n = 1$$

Step 03 : Boundary Conditions

Solution:

$$T(n) = \Omega(n \lg n)$$

$$T(n) \geq cn \lg n$$

$$\begin{array}{rcl} T(n) & & \\ 2T(\lceil n/2 \rceil) + n & \geq & cn \lg n \\ n = 2 & 2T(\lceil 2/2 \rceil) + 2 & \geq c2 \cdot \lg 2 \\ & 2T(1) + 2 & \geq c2 \cdot 1 \\ & 2 + 2 & \geq 2c \\ & 4 & \geq 2c \\ & 4 & \geq 2c \end{array}$$

True for  $0 < c \leq 2$

CP  
CS  
22  
3





# Analysis and Design of Algorithms



## Merge Sort – Lower Bound

Recurrence:

$$T(n) = 2T(\lceil n/2 \rceil) + n$$

$$T(n) = \Theta(1) \quad n = 1$$

Solution:

$$T(n) = \Omega(n \lg n)$$

$$T(n) \geq cn \lg n$$

Step 03 : Boundary Conditions

$$T(n)$$

$$2T(\lceil n/2 \rceil) + n \geq cn \lg n$$

$$n = 3 \quad 2T(\lceil 3/2 \rceil) + 3 \geq c3 \cdot \lg 3$$

$$2T(2) + 3 \geq c3 \cdot (1.58)$$

$$8 + 3 \geq (4.8) c$$

$$11 \geq (4.8) c$$

$$11 \geq (4.8) c$$

True for  $0 < c \leq 2$

Hence proved  $T(n) \geq cn \lg n$



# *Analysis and Design of Algorithms*



## Recurrence

Substitution Method  
(Sample Exercises)



# Analysis and Design of Algorithms



## Class Exercise

$$T(n) = 2T(\lfloor n/2 \rfloor + 17) + n$$

**Guess Solution:**

$$T(n) = O(n \lg n).$$

$$T(n) \leq cn \lg n$$

**Prove using Mathematical Induction. :**

**Step 01 : Assumption**

$$n = \lfloor n/2 \rfloor + 17$$

$$T(\lfloor n/2 \rfloor + 17) \leq c(\lfloor n/2 \rfloor + 17) \lg (\lfloor n/2 \rfloor + 17)$$

CP  
CS  
22  
3



# Analysis and Design of Algorithms



## Class Exercise

Recurrence:

$$T(n) = 2T(\lfloor n/2 \rfloor + 17) + n$$

Step 02 : Prove for all “n”

$$T(n) \leq 2c(\lfloor n/2 \rfloor + 17) \lg(\lfloor n/2 \rfloor + 17) + n$$

For very large value of “n”, we may ignore 17

$$\begin{aligned} T(n) &\leq 2(c \lfloor n/2 \rfloor \lg(\lfloor n/2 \rfloor)) + n \\ &\leq 2(c \cdot n/2 \lg(n/2)) + n \\ &= cn \lg(n/2) + n \\ &= cn \lg n - cn \lg 2 + n \\ &= cn \lg n - cn + n \\ &\leq cn \lg n \end{aligned}$$

Guess Solution:

$$T(n) = O(n \lg n).$$

$$T(n) \leq cn \lg n$$

$$c \geq 1$$

$$n_0 = 1$$



# Analysis and Design of Algorithms



## Merge Sort

- Divide and conquer strategy is applicable to many computational problems.
- Merge sort is a sorting procedure which uses the concept of Divide and conquer
- It involves following three steps:
  - (Divide:) split  $A$  down the middle into two subsequences, each of size roughly  $n/2$
  - (Conquer:) sort each subsequence by calling merge sort recursively on each.
  - (Combine:) merge the two sorted subsequences into a single sorted list.

CP

CS

22

3



# Analysis and Design of Algorithms



## Merge Sort

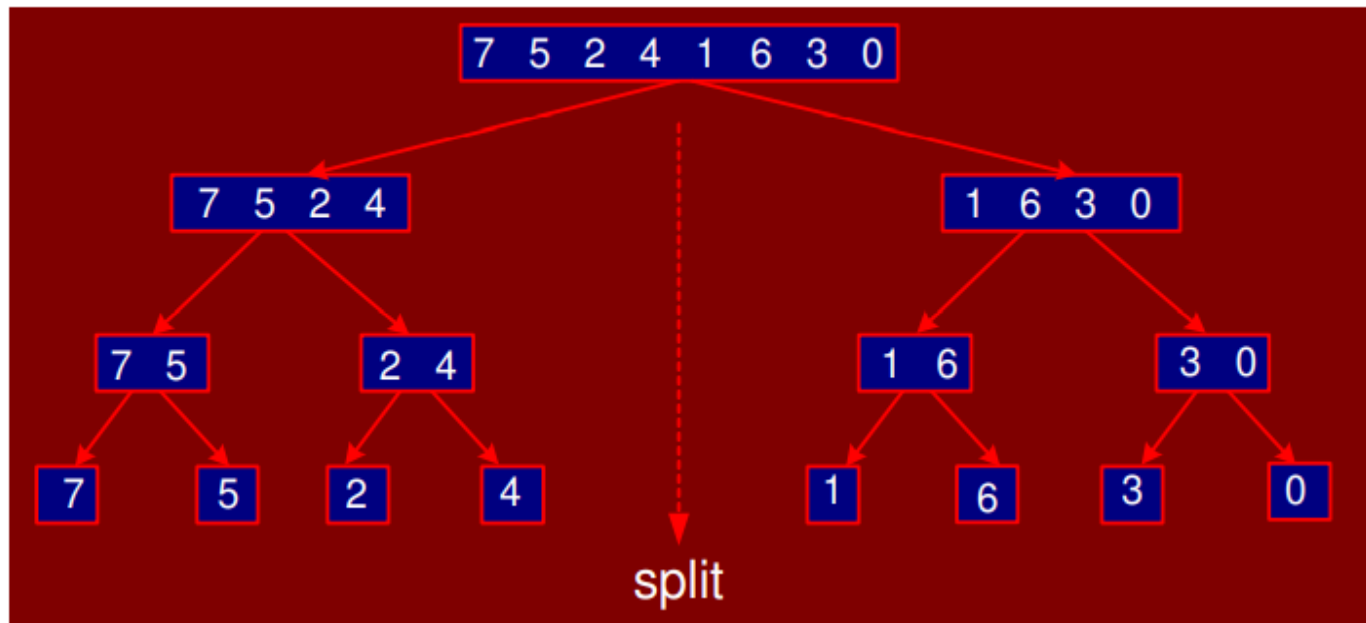
Here is the MergeSort algorithm.

```
MERGE-SORT( array A, int p, int r)
1  if (p < r)
2    then
3       $q \leftarrow (p + r)/2$ 
4      MERGE-SORT(A, p, q)    // sort A[p..q]
5      MERGE-SORT(A, q + 1, r) // sort A[q + 1..r]
6      MERGE(A, p, q, r)    // merge the two pieces
```

CP  
CS  
22  
3

## Merge Sort (Splitting the array)

The following figure illustrates the dividing (splitting) procedure.





# Analysis and Design of Algorithms



## Merge Sort

Here is the merge algorithm:

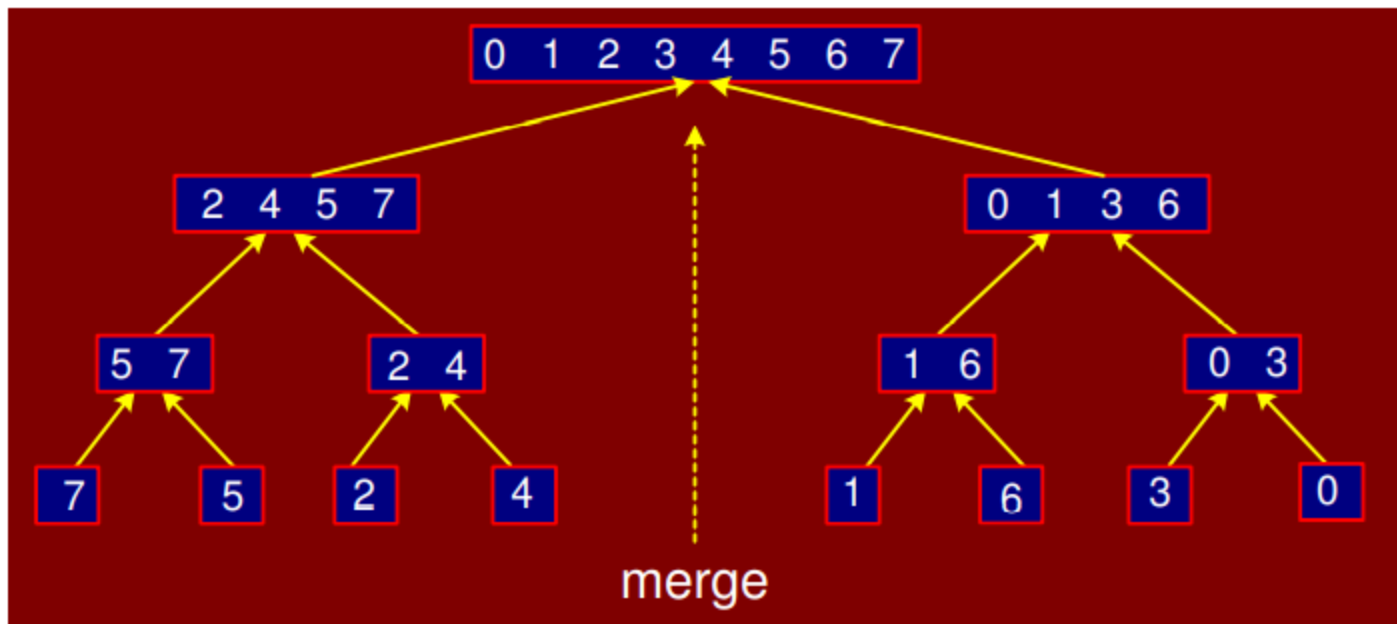
```
MERGE( array A, int p, int q, int r)
1  int B[p..r]; int i ← p; int j ← q + 1
2  while (i ≤ q) and (j ≤ r)
3  do if (A[i] ≤ A[j])
4      then B[i] ← A[i]; i ← i + 1
5      else B[j] ← A[j]; j ← j + 1
6  while (i ≤ q)
7  do B[i] ← A[i]; i ← i + 1
8  while (j ≤ r)
9  do B[j] ← A[j]; j ← j + 1
10 for i ← p to r
11 do A[i] ← B[i]
```

CP  
CS  
22  
3



## Merge Sort (Merging the sub-arrays)

- Here is the merging of sub arrays





# Analysis and Design of Algorithms



## Analysis of Merge Sort

- First of all we shall consider the method  $\text{MERGE}(A, p, q, r)$
- Let  $n = r - p + 1$  be the total length of left and right sub-arrays
- We see that this method contains four loops, none of which is nested
- Each of those loops can be executed at most 'n' times
- Thus running time to merge 'n' items is  $\theta(n)$
- While performing calculations, we will ignore the asymptotic notation ' $\theta$ ' and write it simply as 'n'

CP  
CS  
22  
3



# Analysis and Design of Algorithms



## Analysis of Merge Sort

- Now we shall find the running time of MERGE-SORT method.
- Since, it is a recursive method, it will be defined in terms of recurrence relation.
- Let  $T(n)$  denote the worst case running time of MERGE-SORT on an array of length  $n$ .
- If the array contains only one element, the running time is constant. So, we write it as  $T(n) = 1$  for  $n = 1$  (Base Case)

CP

CS

22

3



# Analysis and Design of Algorithms



## Analysis of Merge Sort

- MERGE-SORT method splits the array into two halves, sorts these two and then merges them together.
- The left half is of size  $\text{ceil}(n/2)$  and the right half is  $\text{floor}(n/2)$ . (If number of array elements 'n' is odd, otherwise it will split it into two equal halves)
- How long does it take to sort elements in sub array of size  $\text{ceil}(n/2)$ ? We do not know this but because  $\text{ceil}(n/2) < n$  for  $n > 1$ , we can express this as  $T(\text{ceil}(n/2))$ . Similarly the time taken to sort right sub array can be expressed as  $T(\text{floor}(n/2))$ .

CP

CS

22

3



# Analysis and Design of Algorithms



## Analysis of Merge Sort

- Hence, the total running time of entire MERGE-SORT method can be expressed as the following recurrence relation:

$$T(n) = \begin{cases} 1 & \text{if } n = 1, \\ T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + n & \text{otherwise} \end{cases}$$



# Analysis and Design of Algorithms



## Analysis of Merge Sort

- Now we shall solve this recurrence relation using **forward substitution** method as follows:

$$T(1) = 1$$

$$T(2) = T(1) + T(1) + 2 = 1 + 1 + 2 = 4$$

$$T(3) = T(2) + T(1) + 3 = 4 + 1 + 3 = 8$$

$$T(4) = T(2) + T(2) + 4 = 4 + 4 + 4 = 12$$

$$T(5) = T(3) + T(2) + 5 = 8 + 4 + 5 = 17$$

...

$$T(8) = T(4) + T(4) + 8 = 12 + 12 + 8 = 32$$

...

$$T(16) = T(8) + T(8) + 16 = 32 + 32 + 16 = 80$$

...

$$T(32) = T(16) + T(16) + 32 = 80 + 80 + 32 = 192$$

CP  
CS  
22  
3



# Analysis and Design of Algorithms



## Analysis of Merge Sort

What is the pattern here? Let's consider the ratios  $T(n)/n$  for powers of 2 only and ignoring others

$$T(1)/1 = 1 \quad T(8)/8 = 4$$

$$T(2)/2 = 2 \quad T(16)/16 = 5$$

$$T(4)/4 = 3 \quad T(32)/32 = 6$$

This suggests  $T(n)/n = \log n + 1$  Or,  $T(n) = n \log n + n$  which is  $\Theta(n \log n)$  (using the limit rule).

CP  
CS  
22  
3



# Analysis and Design of Algorithms



## Analysis of Merge Sort (Second Method-Backward Substitution)

Floor and ceilings are a pain to deal with. If  $n$  is assumed to be a power of 2 ( $2^k = n$ ) (**i.e.  $n$  is even**), this will simplify the recurrence to

$$T(n) = \begin{cases} 1 & \text{if } n = 1, \\ 2T(n/2) + n & \text{otherwise} \end{cases}$$

CP  
CS  
22  
3





# Analysis and Design of Algorithms



## Analysis of Merge Sort (Second Method)

Floor and ceilings are a pain to deal with. If  $n$  is assumed to be a power of 2 ( $2^k = n$ ) (so that each sub-array can have same number of keys), this will simplify the recurrence to

$$T(n) = 2T(n/2) + n$$

$$2^1 T(n/2^1) + 1.n$$

$$= 2(2T(n/4) + n/2) + n$$

$$2^2 T(n/2^2) + 2.n$$

$$= 4T(n/4) + n + n$$

$$= 4(2T(n/8) + n/4) + n + n$$

$$2^3 T(n/2^3) + 3.n$$

$$= 8T(n/8) + n + n + n$$

$$= 8(2T(n/16) + n/8) + n + n + n$$

$$2^4 T(n/2^4) + 4.n$$

$$= 16T(n/16) + n + n + n + n$$

CP  
CS  
22  
3



# Analysis and Design of Algorithms



## Analysis of Merge Sort (Second Method)

If  $n$  is a power of 2 then let  $n = 2^k$  or  $k = \log n$ .

$$\begin{aligned} T(n) &= 2^k T(n/(2^k)) + \underbrace{(n + n + n + \dots + n)}_{k \text{ times}} \\ &= 2^k T(n/(2^k)) + kn \\ &= 2^{(\log n)} T(n/(2^{(\log n)})) + (\log n)n \\ &= 2^{(\log n)} T(n/n) + (\log n)n \\ &= nT(1) + n \log n = n + n \log n \end{aligned}$$

- Hence, running time of MERGE-SORT method is  $T(n) = O(n \log n)$
- For large  $n$ , the number of comparisons made by this algorithm in the average case turns out to be about  $0.25n$  less and hence is also in  $(n \log n)$ .

CP  
CS  
22  
3



# Analysis and Design of Algorithms



## Visualizing Recurrences using the Recurrence Tree

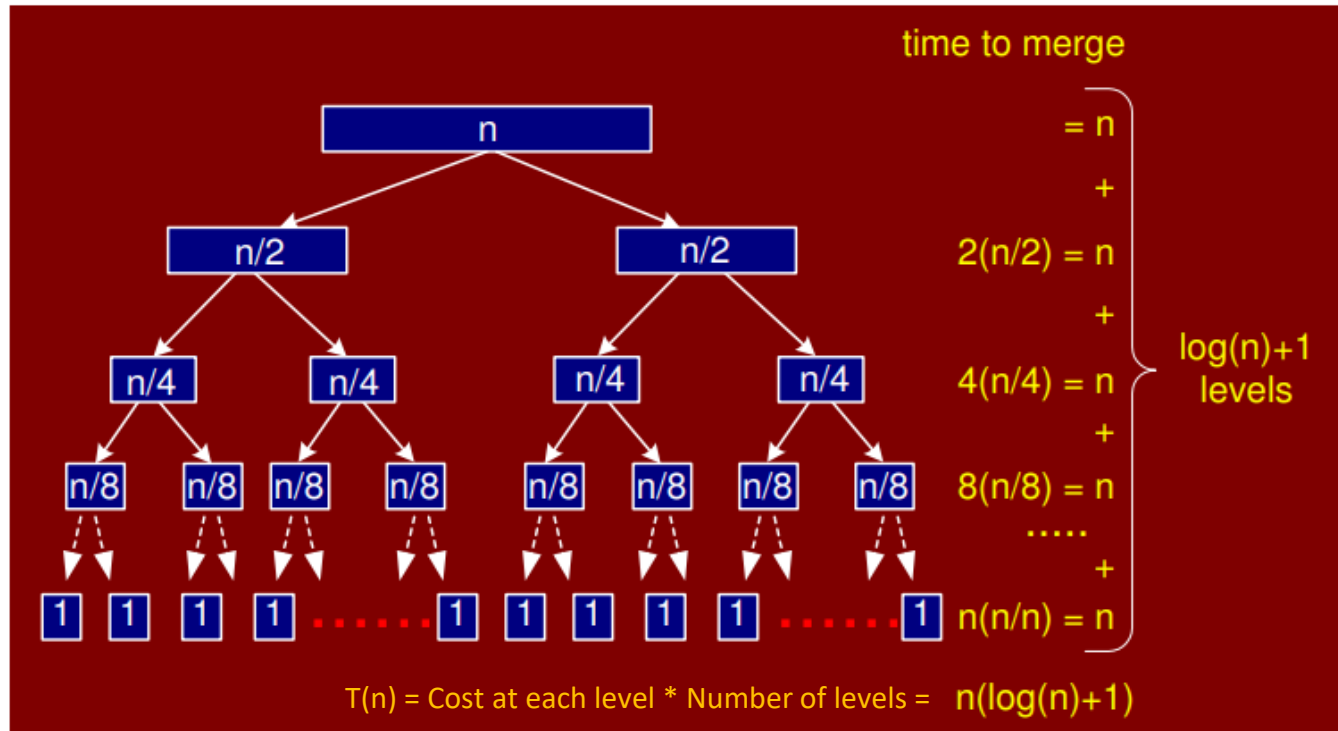
- We can describe any recurrence in terms of a tree, where each expansion of the recurrence takes us one level deeper in the tree.
- We have found that recurrence relation for Merge Sort is

$$T(n) = \begin{cases} 1 & \text{if } n = 1, \\ 2T(n/2) + n & \text{otherwise} \end{cases}$$

- Suppose that we draw the recursion tree for MergeSort, but each time we merge two lists, we label that node of the tree with the time it takes to perform the associated (non-recursive) merge. Recall that to merge two lists of size  $m/2$  to a list of size  $m$  takes  $\theta(m)$  time, which we will just write as  $m$ . Below is an illustration of the resulting recursion tree.
- 

CP  
CS  
22  
3

## Visualizing Recurrences using the Recurrence Tree



### Explanation:

- If  $n=2$ , there will be  $\log_2 2 + 1$  (i.e. 2) levels in the tree
- If  $n=4$ , there will be  $\log_2 4 + 1$  (i.e. 3) levels in the tree
- If  $n=8$ , there will be  $\log_2 8 + 1$  (i.e. 4) levels in the tree



# Analysis and Design of Algorithms



CP  
CS  
22  
3



Allah (Alone) is Sufficient for us, and He is the Best Disposer of affairs (for us). (Quran 3 : 173)